

class-connect- - PDF 3 - frontend/deploy

Source-code export | 2026-09-10 | landscape A4 | monospace | selectable text

Contents:

```
=== frontend/index.html ===  
=== frontend/app.js ===  
=== frontend/_worker.js ===  
=== frontend/sw.js ===  
=== frontend/wrangler.toml ===  
=== README.md ===
```

Line numbers shown at left; long lines wrap with a '+' continuation marker. No secrets are included.

```

1 | <!doctype html>
2 | <html lang="en">
3 | <head>
4 |   <meta charset="utf-8">
5 |   <meta name="viewport" content="width=device-width, initial-scale=1">
6 |   <meta name="theme-color" content="#1F3A34">
7 |   <meta name="description" content="Private classroom communication">
8 |   <title>ClassConnect</title>
9 |   <link rel="manifest" href="/manifest.webmanifest">
10 |   <link rel="icon" href="/icons/icon-192.png">
11 |   <link rel="stylesheet" href="/styles.css">
12 |   <script type="module" src="/app.js"></script>
13 | </head>
14 | <body>
15 |   <section id="auth" class="auth">
16 |     <header>
17 |       <span class="eyebrow">YOUR CLASSROOM, CONNECTED</span>
18 |       <h1>ClassConnect</h1>
19 |       <p>A private space for your class.</p>
20 |     </header>
21 |
22 |     <div class="auth-grid">
23 |       <form id="login-form">
24 |         <h2>Welcome back</h2>
25 |         <label>
26 |           Account recovery key
27 |           <input name="key" type="password" required
28 |             autocomplete="current-password" minlength="64" maxlength="64">
29 |         </label>
30 |         <button class="primary">Sign in</button>
31 |         <p class="muted">Your teacher account is provisioned by the school administrator.</p>
32 |       </form>
33 |
34 |       <form id="enroll-form">
35 |         <h2>First time here?</h2>
36 |         <label>
37 |           Your display name
38 |           <input name="name" required maxlength="60" autocomplete="nickname">
39 |         </label>
40 |         <label>
41 |           Class invitation code
42 |           <input name="code" required maxlength="100" autocomplete="off">
43 |         </label>
44 |         <button class="primary">Join class</button>
45 |         <p class="muted">Already have an account? Sign in first, then join another class.</p>
46 |       </form>
47 |     </div>
48 |   </section>
49 |
50 |   <section id="workspace" class="workspace" hidden>
51 |     <aside class="rail">
52 |       <div class="brand">ClassConnect<span>.</span></div>
53 |       <p id="identity"></p>
54 |
55 |       <div class="rail-actions">
56 |         <button id="join-class">Join another class</button>
57 |         <button id="create-class" hidden>Create class</button>

```

```

58 |     </div>
59 |
60 |     <h2 class="eyebrow">YOUR CLASSES</h2>
61 |     <nav id="class-list" aria-label="Classes"></nav>
62 |
63 |     <div class="rail-bottom">
64 |         <p>Invite-only · 30-day message retention</p>
65 |         <button id="logout">Sign out</button>
66 |     </div>
67 | </aside>
68 |
69 | <main class="conversation">
70 |     <header class="room-header">
71 |         <div>
72 |             <span class="eyebrow">CLASSROOM</span>
73 |             <h1 id="room-title">Choose a class</h1>
74 |         </div>
75 |         <span id="connection" role="status">Not connected</span>
76 |     </header>
77 |
78 |     <div class="transcript-controls">
79 |         <button id="older" disabled>Load older messages</button>
80 |     </div>
81 |
82 |     <section id="messages" class="messages"
83 |         aria-label="Class messages" tabindex="0"></section>
84 |
85 |     <div id="typing" class="typing" aria-live="polite"></div>
86 |     <div id="pending" class="pending" aria-live="polite"></div>
87 |
88 |     <form id="composer" class="composer">
89 |         <label class="sr-only" for="message-text">Message</label>
90 |         <textarea id="message-text" rows="3" maxlength="4000"
91 |             placeholder="Write to your class..." disabled></textarea>
92 |
93 |         <div id="attachment-label" class="muted"></div>
94 |
95 |         <div class="composer-toolbar">
96 |             <label class="file-button">
97 |                 Attach
98 |                 <input id="file" type="file"
99 |                     accept="image/jpeg,image/png,image/webp,application/pdf,audio/webm,audio/ogg"
100 |                     disabled>
101 |             </label>
102 |             <button id="record" type="button" disabled>Record voice</button>
103 |             <button id="cancel-attachment" type="button" hidden>Remove attachment</button>
104 |             <button id="send" class="primary" disabled>Send</button>
105 |         </div>
106 |     </form>
107 | </main>
108 |
109 | <aside class="context">
110 |     <h2>Class roster</h2>
111 |     <p class="muted">Only members of this class can access its messages.</p>
112 |     <button id="rotate-invite" hidden>Rotate invitation code</button>
113 |     <div id="roster"></div>
114 |
115 |     <section class="notice">
116 |         <strong>Privacy note</strong>

```

```
117 |         <p>Messages are not end-to-end encrypted. Your class teacher can moderate them.</p>
118 |     </section>
119 | </aside>
120 | </section>
121 |
122 | <dialog id="secret-dialog">
123 |     <h2 id="secret-title"></h2>
124 |     <p id="secret-explanation"></p>
125 |     <textarea id="secret-value" readonly rows="3"></textarea>
126 |     <button id="copy-secret">Copy</button>
127 |     <button id="close-secret" class="primary">I have saved it</button>
128 | </dialog>
129 |
130 | <div id="toast" role="status" hidden></div>
131 | </body>
132 | </html>
```

=== frontend/app.js === (843 lines, 22427 bytes)

```
1 | const $ = selector => document.querySelector(selector);
2 |
3 | const state = {
4 |   user: null,
5 |   classes: [],
6 |   room: null,
7 |   socket: null,
8 |   generation: 0,
9 |   retry: 0,
10 |   reconnectTimer: null,
11 |   messages: new Map(),
12 |   pending: new Map(),
13 |   roster: [],
14 |   online: new Set(),
15 |   attachment: null,
16 |   uploading: false,
17 |   recorder: null,
18 |   recordingStream: null,
19 |   recordingTimer: null,
20 |   lastTyping: 0,
21 |   lastRead: 0,
22 |   ready: false
23 | };
24 |
25 | function element(tag, text, className) {
26 |   const node = document.createElement(tag);
27 |   if (text !== undefined) node.textContent = text;
28 |   if (className) node.className = className;
29 |   return node;
30 | }
31 |
32 | function button(text, action, className) {
33 |   const node = element("button", text, className);
34 |   node.type = "button";
35 |   node.addEventListener("click", () => run(action));
36 |   return node;
37 | }
38 |
39 | function toast(message) {
40 |   $("#toast").textContent = message;
41 |   $("#toast").hidden = false;
42 |   clearTimeout(toast.timer);
43 |   toast.timer = setTimeout(() => { $("#toast").hidden = true; }, 6500);
44 | }
45 |
46 | async function run(work) {
47 |   try {
48 |     await work();
49 |   } catch (error) {
50 |     toast(error.message || "Something went wrong.");
51 |   }
52 | }
53 |
54 | async function api(path, options = {}) {
55 |   const response = await fetch(`/api${path}`, {
56 |     credentials: "same-origin",
57 |     cache: "no-store",
```

```

58 |     ...options,
59 |     headers: {
60 |       ...(options.body && !(options.body instanceof Blob)
61 |         ? { "Content-Type": "application/json" }
62 |         : {}),
63 |       ...options.headers
64 |     }
65 |   });
66 |
67 |   const data = await response.json().catch(() => ({}));
68 |
69 |   if (!response.ok) {
70 |     const error = new Error(data.error || `Request failed (${response.status}).`);
71 |     error.status = response.status;
72 |     throw error;
73 |   }
74 |
75 |   return data;
76 | }
77 |
78 | function post(path, body = {}) {
79 |   return api(path, { method: "POST", body: JSON.stringify(body) });
80 | }
81 |
82 | function showSecret(title, explanation, value) {
83 |   $("#secret-title").textContent = title;
84 |   $("#secret-explanation").textContent = explanation;
85 |   $("#secret-value").value = value;
86 |   $("#secret-dialog").showModal();
87 | }
88 |
89 | $("#copy-secret").onclick = () => run(async () => {
90 |   await navigator.clipboard.writeText($("#secret-value").value);
91 |   toast("Copied.");
92 | });
93 |
94 | $("#close-secret").onclick = () => {
95 |   $("#secret-dialog").close();
96 |   $("#secret-value").value = "";
97 | };
98 |
99 | $("#secret-dialog").addEventListener("cancel", event => event.preventDefault());
100 |
101 | $("#login-form").onsubmit = event => {
102 |   event.preventDefault();
103 |
104 |   run(async () => {
105 |     const form = new FormData(event.currentTarget);
106 |     const result = await post("/auth/login", { key: form.get("key") });
107 |     $("#login-form").reset();
108 |     await enter(result.user);
109 |   });
110 | };
111 |
112 | $("#enroll-form").onsubmit = event => {
113 |   event.preventDefault();
114 |
115 |   run(async () => {
116 |     const form = new FormData(event.currentTarget);

```

```

117 |     const result = await post("/auth/join", {
118 |       name: form.get("name"),
119 |       code: form.get("code")
120 |     });
121 |
122 |     $("#enroll-form").reset();
123 |
124 |     // Show the only copy before any follow-up network request can fail.
125 |     showSecret(
126 |       "Save your account recovery key",
127 |       "This key signs you in on another device. Store it privately. Your name and invitation code cannot recover this account.",
128 |       result.recoveryKey
129 |     );
130 |
131 |     await enter(result.user);
132 |   });
133 | };
134 |
135 | async function enter(user) {
136 |   state.user = user;
137 |   $("#auth").hidden = true;
138 |   $("#workspace").hidden = false;
139 |   $("#identity").textContent = `${user.display_name} · ${user.role}`;
140 |   $("#create-class").hidden = user.role !== "teacher";
141 |   await loadClasses();
142 | }
143 |
144 | async function loadClasses() {
145 |   const data = await api("/classes");
146 |   state.classes = data.classes;
147 |
148 |   const list = $("#class-list");
149 |   list.replaceChildren();
150 |
151 |   for (const classroom of state.classes) {
152 |     const node = button(classroom.name, () => openRoom(classroom), "class-tab");
153 |     if (state.room?.id === classroom.id) node.classList.add("selected");
154 |     list.append(node);
155 |   }
156 |
157 |   if (!state.room && state.classes.length) {
158 |     await openRoom(state.classes[0]);
159 |   }
160 | }
161 |
162 | $("#create-class").onclick = () => run(async () => {
163 |   const name = prompt("Class name");
164 |   if (!name?.trim()) return;
165 |
166 |   const result = await post("/classes", { name });
167 |
168 |   showSecret(
169 |     "Class invitation code",
170 |     "Share this only with your students. The server stores a hash, not the readable code.",
171 |     result.code
172 |   );
173 |
174 |   await loadClasses();
175 | });

```

```

176 |
177 | $("#join-class").onclick = () => run(async () => {
178 |   const code = prompt("Class invitation code");
179 |   if (!code?.trim()) return;
180 |
181 |   await post("/classes/join", { code });
182 |   await loadClasses();
183 | });
184 |
185 | function stopRecording(discard = false) {
186 |   if (state.recorder?.state === "recording") {
187 |     state.recorder.discard = discard;
188 |     state.recorder.stop();
189 |   }
190 |
191 |   clearTimeout(state.recordingTimer);
192 |   state.recordingStream?.getTracks().forEach(track => track.stop());
193 |   state.recordingStream = null;
194 | }
195 |
196 | $("#logout").onclick = () => run(async () => {
197 |   if (state.pending.size && !confirm("Unsent messages will be discarded. Sign out?")) {
198 |     return;
199 |   }
200 |
201 |   await post("/auth/logout");
202 |   state.generation++;
203 |   clearTimeout(state.reconnectTimer);
204 |   stopRecording(true);
205 |   state.socket?.close();
206 |   location.reload();
207 | });
208 |
209 | function updateComposer() {
210 |   const disabled = !state.room || state.uploading;
211 |   $("#message-text").disabled = disabled;
212 |   $("#file").disabled = disabled;
213 |   $("#record").disabled = disabled;
214 |   $("#send").disabled = disabled || !state.ready;
215 |
216 |   $("#attachment-label").textContent = state.uploading
217 |     ? "Uploading..."
218 |     : state.attachment
219 |       ? `Attached: ${state.attachment.file_name}`
220 |       : "";
221 |
222 |   $("#cancel-attachment").hidden = !state.attachment;
223 | }
224 |
225 | async function openRoom(classroom) {
226 |   if (state.pending.size && !confirm("Discard unsent messages and switch classes?")) {
227 |     return;
228 |   }
229 |
230 |   const generation = ++state.generation;
231 |   clearTimeout(state.reconnectTimer);
232 |   stopRecording(true);
233 |   state.socket?.close();
234 |

```

```

235 | state.room = classroom;
236 | state.messages.clear();
237 | state.pending.clear();
238 | state.roster = [];
239 | state.online.clear();
240 | state.attachment = null;
241 | state.lastRead = 0;
242 | state.ready = false;
243 | state.retry = 0;
244 |
245 | $("#message-text").value = "";
246 | $("#room-title").textContent = classroom.name;
247 | $("#typing").textContent = "";
248 | $("#connection").textContent = "Connecting...";
249 | $("#rotate-invite").hidden = classroom.teacher_id !== state.user.id;
250 | $("#older").disabled = false;
251 |
252 | updateComposer();
253 | renderMessages();
254 | renderPending();
255 |
256 | for (const node of $("#class-list").children) {
257 |   node.classList.toggle("selected", node.textContent === classroom.name);
258 | }
259 |
260 | await refreshRoster(generation);
261 | if (generation === state.generation) connect(generation);
262 | }
263 |
264 | async function refreshRoster(generation = state.generation) {
265 |   if (!state.room) return;
266 |
267 |   const data = await api(`/classes/${state.room.id}/roster`);
268 |   if (generation !== state.generation) return;
269 |
270 |   state.roster = data.members;
271 |   renderRoster();
272 | }
273 |
274 | function connect(generation) {
275 |   if (generation !== state.generation) return;
276 |
277 |   const url = new URL(`/ws/classes/${state.room.id}`, location.href);
278 |   url.protocol = location.protocol === "https:" ? "wss:" : "ws:";
279 |
280 |   const socket = new WebSocket(url);
281 |   state.socket = socket;
282 |
283 |   socket.onmessage = event => {
284 |     if (generation !== state.generation) return;
285 |
286 |     run(async () => {
287 |       const data = JSON.parse(event.data);
288 |
289 |       if (data.type === "ready") {
290 |         state.retry = 0;
291 |         $("#connection").textContent = "Syncing...";
292 |
293 |         // Refresh the loaded window as well as newer messages so deletions

```

```

294 | // made while offline are reconciled.
295 | await synchronize(generation);
296 | await refreshRoster(generation);
297 |
298 | if (generation !== state.generation) return;
299 |
300 | state.ready = true;
301 | $("#connection").textContent = "Connected";
302 | updateComposer();
303 |
304 | for (const item of state.pending.values()) transmit(item);
305 | markRead();
306 | return;
307 | }
308 |
309 | if (data.type === "presence") {
310 |   state.online = new Set(data.userIds);
311 |   renderRoster();
312 |   return;
313 | }
314 |
315 | if (data.type === "typing" && data.userId !== state.user.id) {
316 |   $("#typing").textContent = `${data.name} is typing...`;
317 |   clearTimeout(connect.typingTimer);
318 |   connect.typingTimer = setTimeout(() => {
319 |     $("#typing").textContent = "";
320 |   }, 5000);
321 |   return;
322 | }
323 |
324 | if (data.type === "message:new" || data.type === "message:ack") {
325 |   const message = data.message;
326 |   state.messages.set(message.id, message);
327 |
328 |   if (message.sender_id === state.user.id) {
329 |     state.pending.delete(message.client_id);
330 |   }
331 |
332 |   renderMessages();
333 |   renderPending();
334 |   markRead();
335 |   return;
336 | }
337 |
338 | if (data.type === "message:deleted") {
339 |   const message = state.messages.get(data.id);
340 |   if (message) {
341 |     message.deleted = true;
342 |     message.content = "";
343 |     message.attachment = null;
344 |     renderMessages();
345 |   }
346 |   return;
347 | }
348 |
349 | if (data.type === "message:read") {
350 |   const member = state.roster.find(item => item.id === data.userId);
351 |   if (member) member.last_read_seq = Math.max(member.last_read_seq, data.seq);
352 |   renderMessages();

```

```

353 |     return;
354 | }
355 |
356 | if (data.type === "roster:changed") {
357 |     await refreshRoster(generation);
358 |     return;
359 | }
360 |
361 | if (data.type === "error") {
362 |     if (data.clientId && state.pending.has(data.clientId)) {
363 |         const item = state.pending.get(data.clientId);
364 |         item.status = data.message;
365 |         renderPending();
366 |     }
367 |     toast(data.message);
368 | }
369 | });
370 | };
371 |
372 | socket.onclose = event => {
373 |     if (generation !== state.generation) return;
374 |
375 |     state.ready = false;
376 |     updateComposer();
377 |
378 |     if (event.code === 4001 || event.code === 4003) {
379 |         $("#connection").textContent = event.reason || "Access unavailable";
380 |         toast(event.reason || "Please sign in again.");
381 |         return;
382 |     }
383 |
384 |     $("#connection").textContent = "Disconnected · reconnecting";
385 |
386 |     const delay =
387 |         Math.min(30000, 1000 * 2 ** state.retry++) + Math.random() * 500;
388 |
389 |     state.reconnectTimer = setTimeout(() => connect(generation), delay);
390 | };
391 |
392 | socket.onerror = () => socket.close();
393 | }
394 |
395 | async function synchronize(generation) {
396 |     const existing = [...state.messages.values()];
397 |     let after = existing.length
398 |         ? Math.min(...existing.map(message => message.seq)) - 1
399 |         : null;
400 |
401 |     if (after === null) {
402 |         const data = await api(`/classes/${state.room.id}/messages`);
403 |         if (generation !== state.generation) return;
404 |
405 |         for (const message of data.messages) {
406 |             state.messages.set(message.id, message);
407 |             if (message.sender_id === state.user.id) {
408 |                 state.pending.delete(message.client_id);
409 |             }
410 |         }
411 |     } else {

```

```

412 |     while (true) {
413 |         const data = await api(`/classes/${state.room.id}/messages?after=${after}`);
414 |         if (generation !== state.generation) return;
415 |
416 |         for (const message of data.messages) {
417 |             state.messages.set(message.id, message);
418 |             if (message.sender_id === state.user.id) {
419 |                 state.pending.delete(message.client_id);
420 |             }
421 |         }
422 |
423 |         if (data.messages.length < 100) break;
424 |         after = data.messages.at(-1).seq;
425 |     }
426 | }
427 |
428 | renderMessages(true);
429 | renderPending();
430 | }
431 |
432 | $("#older").onclick = () => run(async () => {
433 |     const messages = [...state.messages.values()];
434 |     if (!messages.length) return;
435 |
436 |     const generation = state.generation;
437 |     const before = Math.min(...messages.map(message => message.seq));
438 |     const data = await api(`/classes/${state.room.id}/messages?before=${before}`);
439 |
440 |     if (generation !== state.generation) return;
441 |
442 |     for (const message of data.messages) {
443 |         state.messages.set(message.id, message);
444 |     }
445 |
446 |     $("#older").disabled = data.messages.length < 100;
447 |     renderMessages(false);
448 | });
449 |
450 | function renderMessages(forceBottom = false) {
451 |     const container = $("#messages");
452 |     const nearBottom =
453 |         container.scrollHeight - container.scrollTop - container.clientHeight < 100;
454 |
455 |     const previousTop = container.scrollTop;
456 |     container.replaceChildren();
457 |
458 |     const messages = [...state.messages.values()].sort((a, b) => a.seq - b.seq);
459 |     let previousSender = null;
460 |     let previousTime = 0;
461 |     let previousDay = "";
462 |
463 |     for (const message of messages) {
464 |         const date = new Date(message.created_at);
465 |         const day = date.toLocaleDateString();
466 |
467 |         if (day !== previousDay) {
468 |             container.append(element("div", day, "day-rule"));
469 |             previousDay = day;
470 |             previousSender = null;

```

```

471 | }
472 |
473 | const row = element("article", undefined, "message");
474 | const grouped =
475 |   previousSender === message.sender_id &&
476 |   message.created_at - previousTime < 5 * 60000;
477 |
478 | if (!grouped) {
479 |   const header = element("header", undefined, "message-header");
480 |   header.append(
481 |     element("strong", message.sender_name),
482 |     element(
483 |       "time",
484 |       date.toLocaleTimeString([], { hour: "2-digit", minute: "2-digit" }),
485 |       "muted"
486 |     )
487 |   );
488 |   row.append(header);
489 | }
490 |
491 | row.append(
492 |   element(
493 |     "p",
494 |     message.deleted ? "Message removed by teacher." : message.content,
495 |     message.deleted ? "muted" : "message-content"
496 |   )
497 | );
498 |
499 | if (message.attachment && !message.deleted) {
500 |   const attachment = message.attachment;
501 |   const url = `/api/files/${attachment.id}`;
502 |
503 |   const link = element(
504 |     "a",
505 |     `${attachment.file_name} · ${Math.ceil(attachment.size_bytes / 1024)} KB`,
506 |     "attachment"
507 |   );
508 |   link.href = url;
509 |   link.download = attachment.file_name;
510 |   row.append(link);
511 |
512 |   if (attachment.mime_type.startsWith("audio/")) {
513 |     const audio = document.createElement("audio");
514 |     audio.controls = true;
515 |     audio.preload = "none";
516 |     audio.src = url;
517 |     row.append(audio);
518 |   }
519 | }
520 |
521 | if (message.sender_id === state.user.id && !message.deleted) {
522 |   const seen = state.roster.filter(member =>
523 |     member.id !== state.user.id && member.last_read_seq >= message.seq
524 |   ).length;
525 |
526 |   row.append(element("small", seen ? `Saved · seen by ${seen}` : "Saved", "muted"));
527 | }
528 |
529 | if (state.room.teacher_id === state.user.id && !message.deleted) {

```

```

530 |         row.append(button("Delete", async () => {
531 |             if (!confirm("Remove this message from the class?")) return;
532 |
533 |             await api(`/classes/${state.room.id}/messages/${message.id}`, {
534 |                 method: "DELETE"
535 |             });
536 |         }, "quiet"));
537 |     }
538 |
539 |     container.append(row);
540 |     previousSender = message.sender_id;
541 |     previousTime = message.created_at;
542 | }
543 |
544 | if (!messages.length) {
545 |     container.append(element("p", "Your classroom conversation starts here.", "empty"));
546 | }
547 |
548 | if (forceBottom || nearBottom) {
549 |     container.scrollTop = container.scrollHeight;
550 | } else {
551 |     container.scrollTop = previousTop;
552 | }
553 | }
554 |
555 | function transmit(item) {
556 |     if (!state.ready || state.socket?.readyState !== WebSocket.OPEN) {
557 |         item.status = "Waiting for connection";
558 |         renderPending();
559 |         return;
560 |     }
561 |
562 |     item.status = "Saving...";
563 |     state.socket.send(JSON.stringify({
564 |         type: "message:new",
565 |         clientId: item.clientId,
566 |         content: item.content,
567 |         attachmentId: item.attachmentId
568 |     }));
569 |
570 |     renderPending();
571 | }
572 |
573 | function renderPending() {
574 |     const container = $("#pending");
575 |     container.replaceChildren();
576 |
577 |     for (const item of state.pending.values()) {
578 |         const row = element("div", undefined, "pending-row");
579 |         row.append(element(
580 |             "span",
581 |             `${item.content} || "Attachment" - ${item.status}`
582 |         ));
583 |
584 |         row.append(button("Retry", () => transmit(item)));
585 |         row.append(button("Discard", () => {
586 |             state.pending.delete(item.clientId);
587 |             renderPending();
588 |         }));

```

```

589 |
590 |     container.append(row);
591 |   }
592 | }
593 |
594 | $("#composer").onsubmit = event => {
595 |   event.preventDefault();
596 |
597 |   const content = $("#message-text").value.trim();
598 |   if (!content && !state.attachment) return;
599 |   if (state.uploading) return;
600 |
601 |   if (state.pending.size >= 10) {
602 |     toast("Resolve your pending messages before sending more.");
603 |     return;
604 |   }
605 |
606 |   const item = {
607 |     clientId: crypto.randomUUID(),
608 |     content,
609 |     attachmentId: state.attachment?.id || null,
610 |     status: "Waiting for connection"
611 |   };
612 |
613 |   state.pending.set(item.clientId, item);
614 |   state.attachment = null;
615 |   $("#message-text").value = "";
616 |   updateComposer();
617 |   transmit(item);
618 | };
619 |
620 | $("#message-text").addEventListener("input", () => {
621 |   if (
622 |     state.ready &&
623 |     state.socket?.readyState === WebSocket.OPEN &&
624 |     Date.now() - state.lastTyping > 3000
625 |   ) {
626 |     state.lastTyping = Date.now();
627 |     state.socket.send(JSON.stringify({ type: "typing" }));
628 |   }
629 | });
630 |
631 | function markRead() {
632 |   const container = $("#messages");
633 |   const nearBottom =
634 |     container.scrollHeight - container.scrollTop - container.clientHeight < 120;
635 |
636 |   if (!state.ready || document.hidden || !nearBottom) return;
637 |
638 |   const seq = Math.max(0, ...[...state.messages.values()].map(message => message.seq));
639 |   if (!seq || seq <= state.lastRead) return;
640 |
641 |   state.lastRead = seq;
642 |   state.socket.send(JSON.stringify({ type: "message:read", seq }));
643 | }
644 |
645 | $("#messages").addEventListener("scroll", () => {
646 |   clearTimeout(markRead.timer);
647 |   markRead.timer = setTimeout(markRead, 500);

```

```

648 | });
649 |
650 | document.addEventListener("visibilitychange", markRead);
651 |
652 | async function uploadFile(file) {
653 |   if (!state.room) return;
654 |   if (file.size > 5 * 1024 * 1024) throw new Error("Maximum attachment size is 5 MiB.");
655 |
656 |   const generation = state.generation;
657 |   const classId = state.room.id;
658 |
659 |   state.uploading = true;
660 |   updateComposer();
661 |
662 |   try {
663 |     const params = new URLSearchParams({
664 |       class: classId,
665 |       name: file.name,
666 |       size: String(file.size)
667 |     });
668 |
669 |     const result = await api(`/uploads?${params}`, {
670 |       method: "POST",
671 |       body: file,
672 |       headers: { "Content-Type": file.type || "application/octet-stream" }
673 |     });
674 |
675 |     if (generation === state.generation) state.attachment = result;
676 |   } finally {
677 |     state.uploading = false;
678 |     updateComposer();
679 |   }
680 | }
681 |
682 | $("#file").onchange = event => run(async () => {
683 |   const file = event.target.files[0];
684 |   event.target.value = "";
685 |   if (file) await uploadFile(file);
686 | });
687 |
688 | $("#cancel-attachment").onclick = () => {
689 |   state.attachment = null;
690 |   updateComposer();
691 | };
692 |
693 | $("#record").onclick = () => run(async () => {
694 |   if (state.recorder?.state === "recording") {
695 |     stopRecording();
696 |     return;
697 |   }
698 |
699 |   if (!navigator.mediaDevices?.getUserMedia || !window.MediaRecorder) {
700 |     throw new Error("Voice recording is unavailable in this browser.");
701 |   }
702 |
703 |   const mime = [
704 |     "audio/webm;codecs=opus",
705 |     "audio/ogg;codecs=opus"
706 |   ].find(type => MediaRecorder.isTypeSupported(type));

```

```

707 |
708 | if (!mime) throw new Error("This browser does not support a compatible audio format.");
709 |
710 | const generation = state.generation;
711 | const stream = await navigator.mediaDevices.getUserMedia({ audio: true });
712 |
713 | if (generation !== state.generation) {
714 |   stream.getTracks().forEach(track => track.stop());
715 |   return;
716 | }
717 |
718 | const chunks = [];
719 | let size = 0;
720 |
721 | const recorder = new MediaRecorder(stream, {
722 |   mimeType: mime,
723 |   audioBitsPerSecond: 32000
724 | });
725 |
726 | state.recordingStream = stream;
727 | state.recorder = recorder;
728 |
729 | recorder.ondataavailable = event => {
730 |   if (event.data.size) {
731 |     chunks.push(event.data);
732 |     size += event.data.size;
733 |     if (size > 4 * 1024 * 1024) stopRecording();
734 |   }
735 | };
736 |
737 | recorder.onerror = () => {
738 |   stopRecording(true);
739 |   toast("Recording failed.");
740 | };
741 |
742 | recorder.onstop = () => {
743 |   stream.getTracks().forEach(track => track.stop());
744 |   clearTimeout(state.recordingTimer);
745 |   $("#record").textContent = "Record voice";
746 |   state.recorder = null;
747 |
748 |   if (recorder.discard || generation !== state.generation) return;
749 |
750 |   run(async () => {
751 |     const extension = mime.includes("ogg") ? "ogg" : "webm";
752 |     const file = new File(chunks, `voice-${Date.now()}.${extension}`, {
753 |       type: mime.split(";")[0]
754 |     });
755 |     await uploadFile(file);
756 |   });
757 | };
758 |
759 | recorder.start(1000);
760 | $("#record").textContent = "Stop recording";
761 | state.recordingTimer = setTimeout(() => stopRecording(), 180000);
762 | });
763 |
764 | function renderRoster() {
765 |   const container = $("#roster");

```

```

766 | container.replaceChildren();
767 |
768 | for (const member of state.roster) {
769 |   const row = element("div", undefined, "roster-member");
770 |   const online = state.online.has(member.id);
771 |
772 |   row.append(
773 |     element("span", online ? "●" : "■", online ? "online" : "muted"),
774 |     element("strong", member.display_name),
775 |     element("small", member.role + (member.muted ? " · muted" : ""), "muted")
776 |   );
777 |
778 |   if (
779 |     state.room?.teacher_id === state.user.id &&
780 |     member.id !== state.user.id
781 |   ) {
782 |     row.append(button(member.muted ? "Unmute" : "Mute", async () => {
783 |       await api(`/classes/${state.room.id}/members/${member.id}`, {
784 |         method: "PATCH",
785 |         body: JSON.stringify({ muted: !member.muted })
786 |       });
787 |     }));
788 |
789 |     row.append(button("Remove", async () => {
790 |       if (!confirm(`Remove ${member.display_name}? The invitation code will also rotate.`)) {
791 |         return;
792 |       }
793 |
794 |       const result = await api(
795 |         `/classes/${state.room.id}/members/${member.id}`,
796 |         { method: "DELETE" }
797 |       );
798 |
799 |       showSecret(
800 |         "New invitation code",
801 |         "The previous code has been invalidated. Share the new code only with approved students.",
802 |         result.code
803 |       );
804 |     }));
805 |   }
806 |
807 |   container.append(row);
808 | }
809 | }
810 |
811 | $("#rotate-invite").onclick = () => run(async () => {
812 |   if (!confirm("Invalidate the existing class invitation code?")) return;
813 |
814 |   const result = await post(`/classes/${state.room.id}/invite`);
815 |
816 |   showSecret(
817 |     "New invitation code",
818 |     "The previous invitation code no longer works.",
819 |     result.code
820 |   );
821 | });
822 |
823 | window.addEventListener("beforeunload", event => {
824 |   if (state.pending.size || state.recorder?.state === "recording") {

```

```
825 |     event.preventDefault();
826 |     event.returnValue = "";
827 |   }
828 | });
829 |
830 | if ("serviceWorker" in navigator) {
831 |   navigator.serviceWorker.register("/sw.js").catch(() => {
832 |     toast("Offline installation support could not be initialized.");
833 |   });
834 | }
835 |
836 | run(async () => {
837 |   try {
838 |     const result = await api("/me");
839 |     await enter(result.user);
840 |   } catch (error) {
841 |     if (error.status !== 401) throw error;
842 |   }
843 | });
```

=== frontend/_worker.js === (49 lines, 1227 bytes)

```
1 | export default {
2 |   async fetch(request, env) {
3 |     const url = new URL(request.url);
4 |
5 |     if (
6 |       url.pathname.startsWith("/api/") ||
7 |       url.pathname.startsWith("/ws/")
8 |     ) {
9 |       return env.API.fetch(request);
10 |    }
11 |
12 |    const response = await env.ASSETS.fetch(request);
13 |    const headers = new Headers(response.headers);
14 |
15 |    headers.set("X-Content-Type-Options", "nosniff");
16 |    headers.set("Referrer-Policy", "no-referrer");
17 |    headers.set("X-Frame-Options", "DENY");
18 |    headers.set(
19 |      "Permissions-Policy",
20 |      "camera=(), microphone=(self), geolocation=()"
21 |    );
22 |    headers.set(
23 |      "Content-Security-Policy",
24 |      [
25 |        "default-src 'self'",
26 |        "script-src 'self'",
27 |        "style-src 'self'",
28 |        "connect-src 'self'",
29 |        "img-src 'self' blob:",
30 |        "media-src 'self' blob:",
31 |        "worker-src 'self'",
32 |        "manifest-src 'self'",
33 |        "object-src 'none'",
34 |        "frame-ancestors 'none'",
35 |        "base-uri 'self'",
36 |        "form-action 'self'"
37 |      ].join("; ")
38 |    );
39 |
40 |    if (url.pathname === "/sw.js") {
41 |      headers.set("Cache-Control", "no-cache");
42 |    }
43 |
44 |    return new Response(response.body, {
45 |      status: response.status,
46 |      headers
47 |    });
48 |  }
49 | };
```

=== frontend/sw.js === (56 lines, 1228 bytes)

```
1 | const CACHE = "classconnect-shell-v1";
2 |
3 | const SHELL = [
4 |   "/",
5 |   "/index.html",
6 |   "/app.js",
7 |   "/styles.css",
8 |   "/manifest.webmanifest",
9 |   "/icons/icon-192.png",
10 |  "/icons/icon-512.png"
11 | ];
12 |
13 | self.addEventListener("install", event => {
14 |   event.waitUntil(caches.open(CACHE).then(cache => cache.addAll(SHELL)));
15 | });
16 |
17 | self.addEventListener("activate", event => {
18 |   event.waitUntil(async () => {
19 |     const names = await caches.keys();
20 |
21 |     await Promise.all(
22 |       names
23 |         .filter(name => name.startsWith("classconnect-shell-") && name !== CACHE)
24 |         .map(name => caches.delete(name))
25 |     );
26 |
27 |     await self.clients.claim();
28 |   })();
29 | });
30 |
31 | self.addEventListener("fetch", event => {
32 |   const request = event.request;
33 |   const url = new URL(request.url);
34 |
35 |   if (
36 |     request.method !== "GET" ||
37 |     url.origin !== self.location.origin ||
38 |     url.pathname.startsWith("/api/") ||
39 |     url.pathname.startsWith("/ws/")
40 |   ) {
41 |     return;
42 |   }
43 |
44 |   if (request.mode === "navigate") {
45 |     event.respondWith(
46 |       fetch(request).catch(() => caches.match("/index.html"))
47 |     );
48 |     return;
49 |   }
50 |
51 |   if (SHELL.includes(url.pathname)) {
52 |     event.respondWith(
53 |       caches.match(request).then(cached => cached || fetch(request))
54 |     );
55 |   }
56 | });
```

=== frontend/wrangler.toml === (8 lines, 145 bytes)

```
1 | name = "classconnect"
2 | pages_build_output_dir = "."
3 |
4 | compatibility_date = "2025-04-01"
5 |
6 | [[services]]
7 | binding = "API"
8 | service = "classconnect-api"
```

=== README.md === (120 lines, 4703 bytes)

```
1 | # ClassConnect
2 |
3 | Private classroom communication: teachers are provisioned (no public teacher registration), students join a class with an invitation code, and each class gets a persistent
  | + real-time chat with typing, presence, read receipts and attachments.
4 |
5 | ## Stack
6 |
7 | - **Cloudflare Workers** API + **Durable Objects** (real-time rooms, budget guard)
8 | - **Cloudflare D1** (SQLite) with migrations
9 | - **Cloudflare R2** for private attachments (image, PDF, voice notes)
10 | - **Cloudflare Pages** frontend – no bundler; served directly, proxied same-origin to the API (incl. WebSockets) via `frontend/_worker.js`
11 | - **PWA** shell: `frontend/sw.js` + `frontend/manifest.webmanifest` + generated icons
12 |
13 | ## Layout
14 |
15 | ```
16 | backend/           Worker API (src/index.js), wrangler.toml, migrations/, tests
17 | frontend/         Pages site: _worker.js proxy, app.js, styles.css, index.html,
18 |                   sw.js, manifest.webmanifest, icons/
19 | scripts/          make-icons.py, provision-teacher.mjs
20 | .github/workflows/ deploy.yml (manual Cloudflare deployment)
21 | ```
22 |
23 | ## Local development
24 |
25 | Requires Node 22 and Python 3.
26 |
27 | ```bash
28 | npm install
29 |
30 | # local dev config (gitignored) – matches backend/.dev.vars:
31 | # APP_ORIGIN=http://localhost:8788
32 |
33 | npm run db:local      # apply D1 migrations to the local database
34 | npm run dev:api       # API worker on :8787
35 | npm run dev:web       # Pages site on :8788, service-bound to the API
36 | ```
37 |
38 | Provision a teacher (writes `teacher-seed.sql` + `teacher-credentials.txt` – never commit either):
39 |
40 | ```bash
41 | npm run teacher -- "Ms. Rahman"
42 | ```
43 |
44 | Then insert `teacher-seed.sql` into the local D1 database file (`backend/.wrangler/state/v3/d1/.../*.sqlite`) before signing in locally.
45 |
46 | Run checks:
47 |
48 | ```bash
49 | npm test              # 6 validation tests
50 | npm run icons         # regenerate frontend/icons/icon-{192,512}.png
51 | ```
52 |
53 | ## First deployment
54 |
55 | ```bash
56 | npm install && npm run icons && npm test
```

```

57 | npx wrangler login
58 |
59 | npx wrangler d1 create classconnect-db
60 | npx wrangler r2 bucket create classconnect-files
61 | npx wrangler pages project create classconnect --production-branch main
62 | ```
63 |
64 | **Replace the placeholders before deploying:**
65 |
66 | - `backend/wrangler.toml` → `database_id` (from `d1 create`) and `APP_ORIGIN` (your Pages hostname, e.g. `https://classconnect.pages.dev`)
67 | - If the Pages project name was unavailable, also update `package.json` (`deploy:web`) and `frontend/wrangler.toml`
68 | - In the Cloudflare dashboard confirm the Pages production environment has the service binding `API` → `classconnect-api`
69 |
70 | Apply the schema and provision the teacher:
71 |
72 | ```bash
73 | npm run db:remote
74 | npm run teacher -- "Ms. Rahman"
75 | npx wrangler d1 execute classconnect-db --remote --config backend/wrangler.toml --file teacher-seed.sql
76 | ```
77 |
78 | Store the generated recovery key privately, then delete the temporary provisioning files.
79 |
80 | Deploy:
81 |
82 | ```bash
83 | npm run deploy:api
84 | npm run deploy:web
85 | ```
86 |
87 | The GitHub Actions workflow `.github/workflows/deploy.yml` does the same manually (`workflow_dispatch`) using repo secrets `CLOUDFLARE_API_TOKEN` and
  + `CLOUDFLARE_ACCOUNT_ID` – use a least-privilege token scoped to Workers, Pages, D1 and the required storage operations.
88 |
89 | ## What's left / not in this release
90 |
91 | **Deliberately disabled – add after the core is deployed and allowances are confirmed:**
92 |
93 | - Real-time voice/video calls (TURN / SFU) – no fake call buttons
94 | - Background push notifications
95 | - Student-to-student direct messages
96 |
97 | **Remaining work before real student data:**
98 |
99 | - WebSocket + device tests (HTTP API flows are covered by `npm test`), including: failed D1 write → no false acknowledgement; repeated `clientId` → one saved message;
  + removed member → socket closed; logout → later socket operations rejected; reconnect → saved messages reconciled; Android microphone permission denied → recoverable UI
100 | - Verify the Pages service binding (cookie + WebSocket auth) on the actual production hostname
101 | - Check R2 billing conditions in your account; budget alerts are not automatic hard spending stops
102 | - No antivirus scanning: PDFs and attachments must still be treated as untrusted files
103 |
104 | **Enforced boundaries (intentional):**
105 |
106 | | Control | Limit |
107 | | --- | --- |
108 | | Class membership | 40 students + teacher |
109 | | Live sockets per class | 82 |
110 | | Live sockets per account per class | 2 |
111 | | Message length | 4,000 characters |
112 | | WebSocket events per user per class | 90/minute |
113 | | Attachment size | 5 MiB |

```

114 | | Uploads per user | 10/day |
115 | | Upload bytes per user | 20 MiB/day |
116 | | Upload bytes across installation | 50 MiB/day |
117 | | Lifetime accepted upload reservations | 512 MiB (does not reset) |
118 | | Message & attachment retention | 30 days |
119 | | Voice recording | 3 minutes |
120 | | Session lifetime | 7 days |